

Turing Machines

Recap

DFA/NFA : Computational model for devices with limited memory.

PDA : Devices that has unlimited stack memory.

These models have limitations

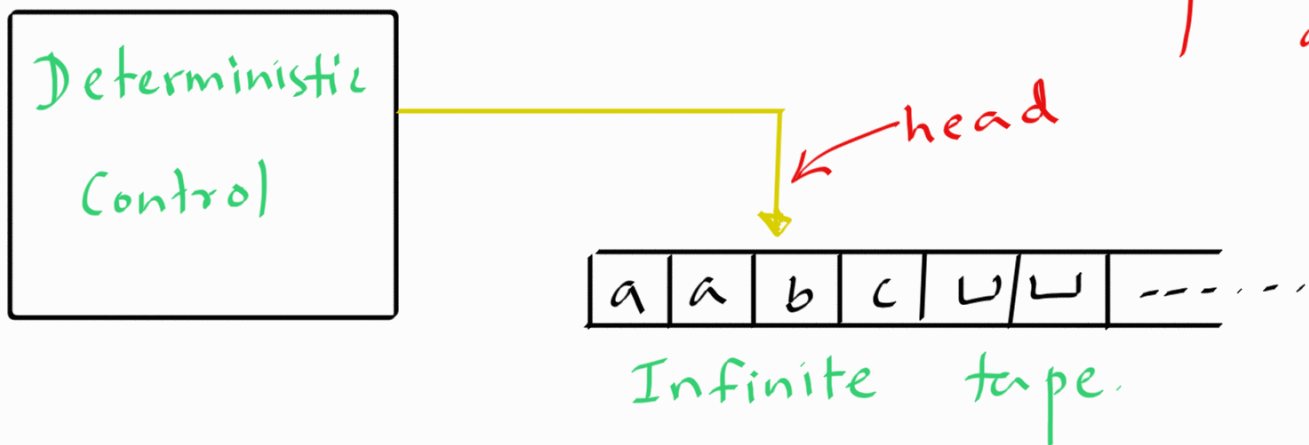
They are too restrictive.

$$A = \{a^n b^n c^n \mid n \geq 0\}$$

Turing Machines

Model for general computers

- has unlimited and unrestricted memory.
- can do anything that a real computer can do.



1. Initially tape contains the input and blank symbol everywhere else.
2. Machine can read/write information on tape.
3. To read information from the tape, machine can move its head both to the left and to the right.
4. Special states for accepting and rejecting take effect immediately.
5. Machine computes until it produces an output.
 - Accept / Reject / loop

Ex: $A = \{0^{2^n} \mid n \geq 0\}$



general programming
language

C, C++, Java, ...

test if # of 0's (x) = 2^n

Alg 0(x):

if $x == 1$ accept

$x \% 2 == 1$, No

$x \% 2 == 0$, recurse on $x/2$

Any function that can be computed
by an intuitive algorithm can be
computed by a Turing machine.

intuitive algorithm

1. A step-by-step finite process
2. Using discrete, finite symbols
3. Running in finite time.

Differences between finite automata and Turing Machines.

1. Turing Machine can both write on tape and read from tape.
2. Read-write head can move both left and right.
3. Tape is infinite.
4. Special states for accepting and rejecting take effect immediately.

Def

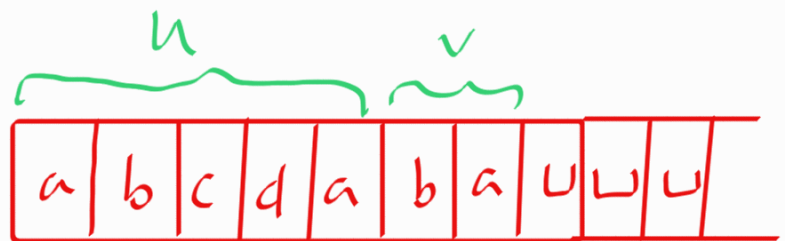
A Turing Machine is a 7-tuple, $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where

1. Q is the set of states.
2. Σ is the input alphabet
3. Γ is the tape alphabet, where
 $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$ $\sqcup \leftarrow$ blank symbol
4. $\delta : Q \times \Gamma \longrightarrow Q \times \Gamma \times \{L, R\}$
5. $q_0 \in Q$ is the start state
6. $q_{\text{accept}} \in Q$ is the accept state
7. $q_{\text{reject}} \in Q$ is the reject state
 $q_{\text{accept}} \neq q_{\text{reject}}$

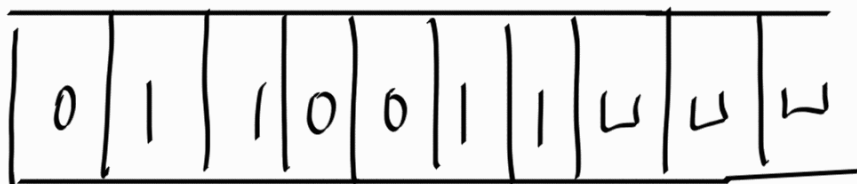
How to represent the state of the machine (configuration)

we have 3 items.

- (i) current state
- (ii) tape content
- (iii) current head location



$u q v$

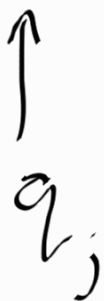
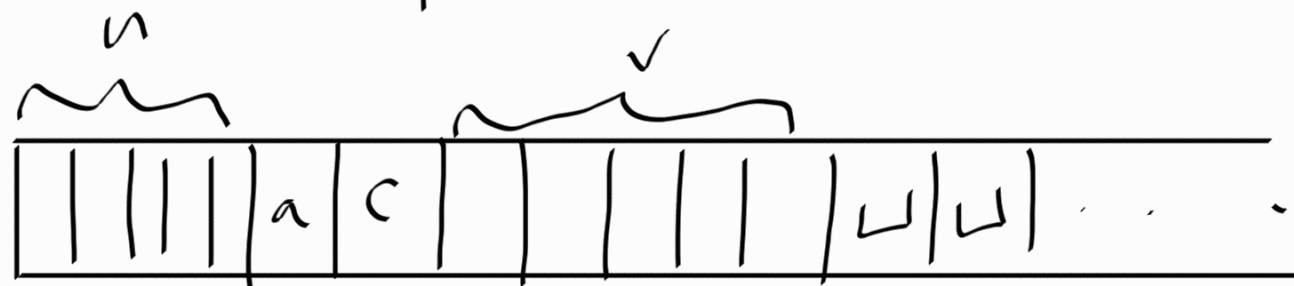
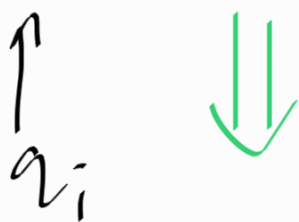
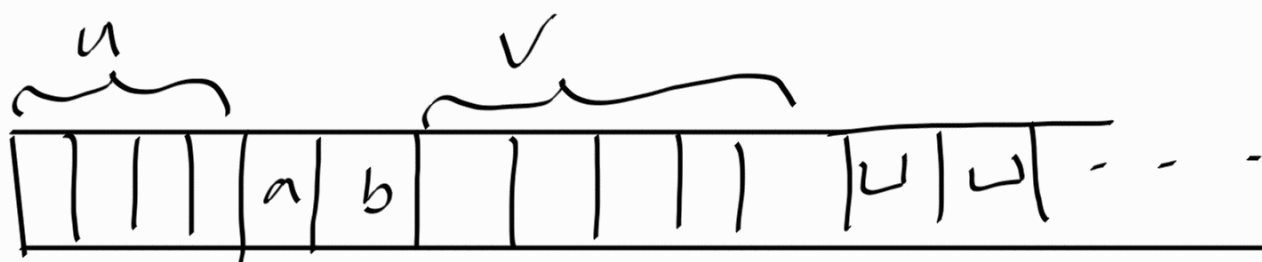


$01100q_211$

we say configuration C_1 yields configuration C_2 if the turing machine can legally go from C_1 to C_2 in a single step.

If we have $\delta(q_i, b) = (q_j, c, L)$

$u a q_i b v$ yields $u q_j a c v$



$u q_j a c v$

What is the start configuration on string w .

$q_0 w$

When you create Turing machine to accept some language, you can describe them using 3 different ways.

1. Formal turing machine description
you draw states and transitions
and create the machine.

2. Implementation level machine
description

you explain how the turing machine
process the input, read and write
to the tape.

3. Higher level description
you mention the algorithm
that can accept a string in your
language.

$$A = \{0^{2^n} \mid n \geq 0\}$$

$M =$ "on input string w "

1. Sweep ^{from} left to right across the tape, crossing off every other zero.
2. If in stage 1, the tape contains a single 0, accept
3. If in stage 1, the tape contains more than single 0 and the # of 0's was odd, reject
4. Return the head to the left hand end of the tape
5. Go to stage 1.

(Implementation level description of a TM)

Note that whenever we mark a 0, we have to see 2 zeros.
∴ when we mark a 0, we know that we have seen even # of zero's.

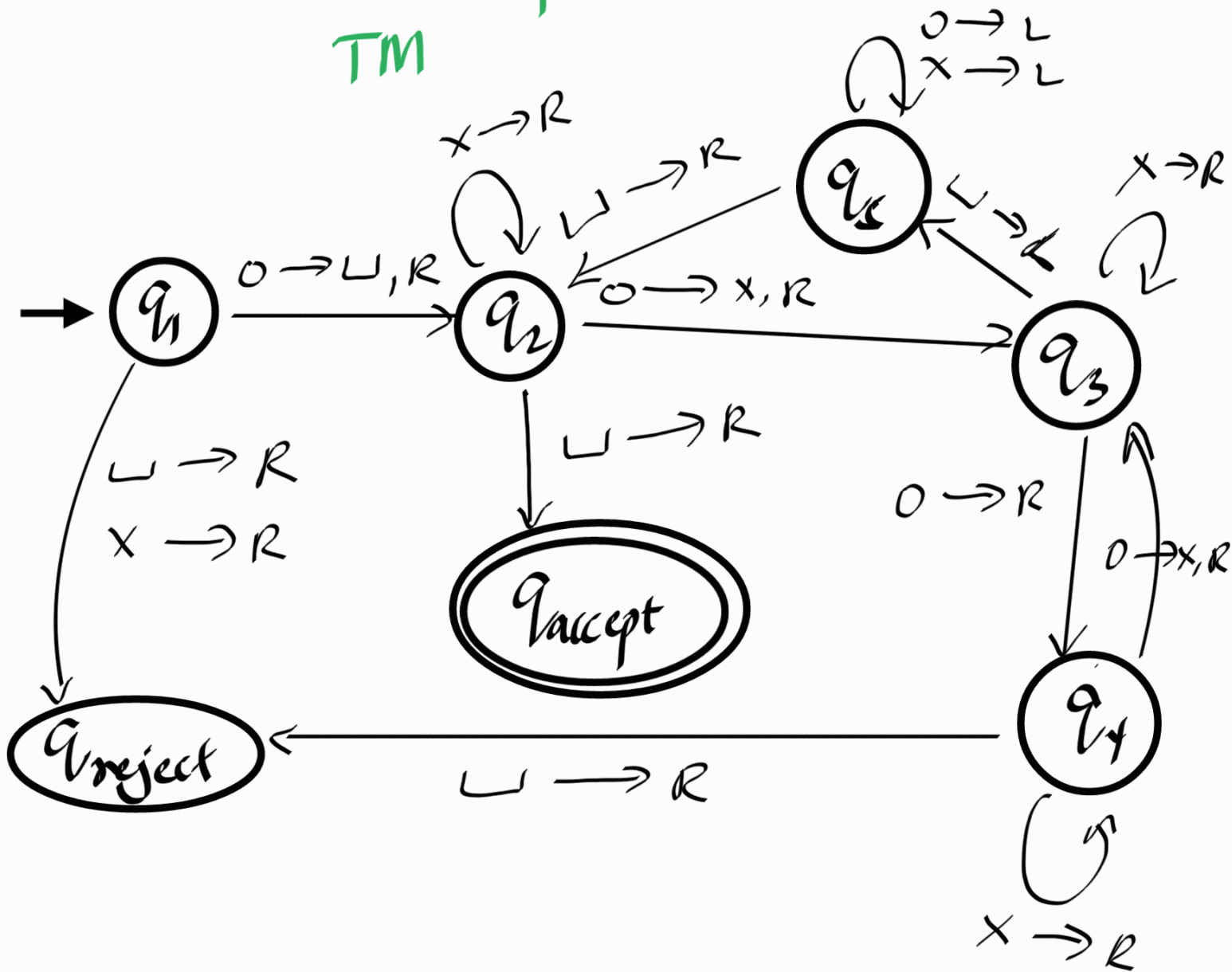
* Each iteration halves the number of 0's.

* If the remaining # of 0's is odd and greater than 1, we reject.

* we do this until we have only one 0 left.

formal description of this

TM



$a \rightarrow L, R$ means

read a from head writ L to stack
and move head to write.

$a \rightarrow L$ means read a from head
don't change tape and move left.

